"# AETERNA MVP - 2-3 Month Development Roadmap

Team Structure & Task Distribution

Project Duration: 12 Weeks (3 Months)
Team Size: 3 Members
Sprint Duration: 2 Weeks

👥 Team Roles & Responsibilities

🔷 Person 1: Backend Developer (You)

Primary Focus: Backend architecture, APIs, data processing, and database

Core Responsibilities:

- FastAPI backend development
- PostgreSQL database design and implementation
- Data ingestion systems (RSS, Twitter, Ethereum, Price APIs)
- Event processing system (Agent A - The Sieve)
- Alert generation logic
- Authentication & authorization
- API endpoints for frontend consumption
- Backend testing and optimization

Tech Stack:

- Python 3.11+
- FastAPI
- PostgreSQL 15+
- Redis 7+
- RabbitMQ 3.11+
- SQLAlchemy
- Celery
- Web3.py
- Python-telegram-bot

🔶 Person 2: Frontend Developer

Primary Focus: User interface, dashboard, and user experience

Core Responsibilities:

- React 18+ application development
- Web dashboard UI/UX implementation
- Landing page and marketing site
- Real-time WebSocket integration
- User authentication flows
- Alert feed and detail views
- Settings and preferences UI
- Responsive design (mobile, tablet, desktop)
- Frontend testing

Tech Stack:

- React 18+
- TailwindCSS
- Socket.io-client
- React Router
- Axios
- React Query
- Modern JavaScript (ES6+)

🔵 Person 3: Integration & DevOps Specialist

Primary Focus: Third-party integrations, deployment, and infrastructure

Core Responsibilities:

- External API integrations (Twitter, Telegram, SendGrid, etc.)
- Telegram bot implementation
- Email service integration (SendGrid)
- Docker containerization
- Cloud deployment (Railway/Render)
- CI/CD pipeline setup
- Monitoring and logging (Sentry, UptimeRobot)
- Database backups and recovery
- Performance optimization
- Security implementation

Tech Stack:

- Docker & Docker Compose

- RabbitMQ
- Redis
- Twitter API v2
- Telegram Bot API
- SendGrid API
- QuickNode/Alchemy (Ethereum)
- CoinGecko API
- GitHub Actions (CI/CD)

📅 Sprint-by-Sprint Breakdown

Phase 1: Foundation & Core Infrastructure (Weeks 1-4)

Sprint 1: Week 1-2 - Project Setup & Core Infrastructure

🔷 **Backend Developer Tasks:**

Priority: P0 (Critical)

Week 1:

- [ ] Set up project repository and structure
- [ ] Initialize FastAPI application with basic config
- [ ] Design database schema for MVP
  - Users table (id, email, password_hash, telegram_id, preferences, created_at, email_verified)
  - Events table (id, event_type, source, content, metadata, score, priority, timestamp)
  - Alerts table (id, user_id, event_id, channels, status, sent_at, created_at)
  - User_preferences table (JSON field for watchlist, frequency, quiet_hours)
- [ ] Set up PostgreSQL database connection with SQLAlchemy
- [ ] Create database models and migrations (Alembic)
- [ ] Implement basic health check endpoint (`/health`)

Week 2:

- [ ] Implement user registration API endpoint
  - Email validation
  - Password hashing (Bcrypt, cost 12)
  - Input validation and error handling
- [ ] Implement user login API endpoint
  - JWT token generation (24-hour expiry)

    - Refresh token mechanism (30-day expiry)
- ☐ Implement password reset flow (token generation)
- ☐ Create user profile endpoints (GET, PATCH)
- ☐ Set up CORS configuration
- ☐ Write unit tests for auth endpoints (70%+ coverage)

Deliverables:

- ✅ FastAPI app with database connection
- ✅ User authentication system (register, login, password reset)
- ✅ Database schema v1.0 with migrations
- ✅ API documentation (OpenAPI/Swagger)

🔶 **Frontend Developer Tasks:**

Priority: P0 (Critical)

Week 1:

- ☐ Set up React 18 project with Vite
- ☐ Configure TailwindCSS and design system
    - Define color palette (Blue, Green, Red, Amber)
    - Set up typography (Inter font family)
    - Create base CSS utilities
- ☐ Set up React Router for navigation
- ☐ Create project folder structure
    - `/components` (reusable UI components)
    - `/pages` (page-level components)
    - `/hooks` (custom React hooks)
    - `/services` (API calls)
    - `/utils` (helper functions)
    - `/contexts` (React Context)
- ☐ Create AuthContext for authentication state management

Week 2:

- ☐ Design and implement Landing Page
    - Hero section with value proposition
    - Features section
    - Pricing section (placeholder)
    - CTA buttons
    - Footer
- ☐ Create Login page

    - Email and password fields
    - Form validation
    - Error message display
    - "Forgot Password" link
- ☐ Create Registration page
    - Email, password, confirm password fields
    - Terms of service checkbox
    - Form validation (client-side)
    - Success state handling
- ☐ Implement authentication flow
    - API integration with backend
    - JWT token storage (localStorage/sessionStorage)
    - Protected route wrapper
    - Auto-redirect logic
- ☐ Create basic layout components (Header, Sidebar, Footer)

Deliverables:

- ✅ React app with routing
- ✅ Landing page (responsive)
- ✅ Authentication pages (Login, Register)
- ✅ Basic layout structure

🔵 **Integration & DevOps Tasks:**

Priority: P0 (Critical)

Week 1:

- ☐ Set up development environment documentation
- ☐ Create Docker Compose configuration
    - PostgreSQL service
    - Redis service
    - RabbitMQ service
    - Backend service
    - Frontend service
- ☐ Set up environment variable management
    - Create `.env.example` files
    - Document all required API keys
- ☐ Configure RabbitMQ
    - Set up exchanges and queues
    - Configure topic exchange for event routing

- ☐ Set up Redis for caching
  - Configure connection pooling
  - Test basic operations

Week 2:

- ☐ Obtain and configure API keys
  - Twitter API v2 (Bearer Token)
  - QuickNode/Alchemy (Ethereum node)
  - CoinGecko API (optional key)
  - Telegram Bot API (create bot, get token)
  - SendGrid API (get API key)
- ☐ Create initial deployment documentation
- ☐ Set up Git workflow and branching strategy
- ☐ Configure pre-commit hooks (linting, formatting)
- ☐ Set up basic monitoring (health check endpoints)

Deliverables:

- ✅ Local development environment with Docker Compose
- ✅ All external API keys obtained and documented
- ✅ Development workflow documentation
- ✅ RabbitMQ and Redis configured

Sprint 2: Week 3-4 - Data Ingestion System

🔷 **Backend Developer Tasks:**

Priority: P0 (Critical)

Week 3:

- ☐ Implement event normalization schema
  - Create unified Event model
  - Timestamp normalization (UTC ISO8601)
  - Unique ID generation strategy
- ☐ Build RSS News Collector
  - Feed parser implementation
  - Target sources: CoinDesk, CoinTelegraph, Decrypt
  - Poll every 60 seconds
  - Normalize to unified schema
  - Publish to RabbitMQ queue
  - Error handling and retry logic (3 attempts)

- ☐ Build Price Data Collector (CoinGecko)
  - Track top 100 cryptocurrencies
  - Poll every 60 seconds
  - Calculate price changes (1h, 24h)
  - Detect significant moves (>5%)
  - Normalize and publish events
- ☐ Implement duplicate detection mechanism
  - Content hashing function
  - Redis cache for 1-hour window
  - <10ms lookup time requirement

Week 4:

- ☐ Create RabbitMQ publisher utility
  - Connection pooling
  - Retry logic
  - Error handling
- ☐ Implement data ingestion service orchestration
  - Celery tasks for each collector
  - Scheduled task execution
  - Health monitoring
- ☐ Create database models for events
- ☐ Implement event storage endpoints
- ☐ Add logging and monitoring for ingestion pipeline
- ☐ Write unit tests for collectors (70%+ coverage)
- ☐ Performance testing (target: 10,000+ events/hour)

Deliverables:

- ✅ RSS News Collector (functional)
- ✅ Price Data Collector (functional)
- ✅ Event normalization pipeline
- ✅ Duplicate detection system
- ✅ RabbitMQ integration
- ✅ 10,000+ events/hour throughput

🔶 **Frontend Developer Tasks:**

Priority: P1 (High)

Week 3:

- ☐ Design Dashboard wireframes and mockups

    - Alert feed layout
    - Filter sidebar
    - Alert card design
    - Detail view modal
- ☐ Create reusable UI components
    - Button component (variants: primary, secondary, danger)
    - Input component (text, email, password)
    - Card component
    - Badge component (priority indicators)
    - Loading spinner
    - Alert/Toast notification
- ☐ Implement dashboard layout
    - Header with user profile dropdown
    - Sidebar for filters
    - Main content area
    - Responsive breakpoints

Week 4:

- ☐ Create Alert Card component
    - Display event type, source, timestamp
    - Priority badge (HIGH/MEDIUM/LOW)
    - Content preview (truncated)
    - Action buttons (view details, mark as read)
    - Hover states
- ☐ Implement Alert Feed view (static data)
    - List of alert cards
    - Infinite scroll setup (react-infinite-scroll-component)
    - Loading states
    - Empty state design
- ☐ Create Filter component
    - Priority filter (HIGH, MEDIUM, LOW)
    - Date range picker
    - Entity/token filter (search input)
    - Apply/Clear buttons
- ☐ Add dark mode support (optional enhancement)

Deliverables:

- ✅ Dashboard UI (static)
- ✅ Reusable component library
- ✅ Alert card design
- ✅ Filter UI

🔵 **Integration & DevOps Tasks:**

Priority: P0 (Critical)

Week 3:

- ☐ Integrate Twitter API v2
  - Set up Filtered Stream API connection
  - Configure filter rules for crypto keywords
    - Keywords: Bitcoin, Ethereum, crypto, blockchain, DeFi, NFT, etc.
  - Handle tweet stream processing
  - Extract relevant metadata (text, author, engagement)
  - WebSocket connection management
  - Auto-reconnect on disconnect
  - Rate limit handling
- ☐ Test Twitter integration
  - Verify 50+ tweets/second handling
  - Latency testing (<100ms)
  - Error scenario testing

Week 4:

- ☐ Integrate Ethereum blockchain monitoring
  - Connect to QuickNode/Alchemy WebSocket
  - Subscribe to new blocks (eth_subscribe)
  - Filter large transactions (>$1M USD)
  - Identify exchange addresses (maintain address list)
  - Handle WebSocket connection failures
  - Implement fallback node switching
- ☐ Test blockchain integration
  - Verify <15 second latency
  - Test failover to backup node
  - Monitor API rate limits
- ☐ Set up monitoring for data collectors
  - Create health check endpoints for each collector
  - Log ingestion rates
  - Alert on failures

Deliverables:

- ✅ Twitter data collector (50+ tweets/sec)
- ✅ Ethereum blockchain monitor (<15s latency)
- ✅ All 4 data sources operational (RSS, Twitter, Ethereum, Price)

- ✅ Monitoring dashboard for ingestion metrics

Phase 2: Intelligence & Processing (Weeks 5-8)

Sprint 3: Week 5-6 - Event Processing System (Agent A)

🔷 **Backend Developer Tasks:**

Priority: P0 (Critical)

Week 5:

- [ ] Implement Agent A - The Sieve (Noise Filter)
  - Create event scoring system
  - Multi-source check (3+ sources = 100 pts)
  - Engagement analysis (rate >5% = 100 pts, verified +20 pts)
  - Bot detection (spam patterns, generic usernames)
  - Semantic similarity detection (cosine similarity >0.9)
  - Weighted score calculation (30% multi-source, 20% engagement, 30% bot, 20% dedup)
- [ ] Implement RabbitMQ consumer for event processing
  - Read from ingestion queue
  - Process through Agent A
  - Assign priority (HIGH ≥80, MEDIUM 50-79, LOW <50)
- [ ] Create processed event storage
  - Store scored events in database
  - 7-day retention policy
  - Index on priority, timestamp, user_id

Week 6:

- [ ] Optimize scoring algorithms
  - Performance tuning (<5 seconds per event)
  - Batch processing optimization
  - Cache frequently accessed data
- [ ] Implement alert generation logic
  - Generate alerts for HIGH and MEDIUM priority events
  - Format for multiple channels (Telegram, Email, Web)
  - User preference filtering
  - Rate limiting (max 10 alerts/hour/user)
  - Quiet hours respect
- [ ] Create alert API endpoints

    - GET /api/alerts (list user's alerts)
    - GET /api/alerts/:id (alert details)
    - PATCH /api/alerts/:id (mark as read)
    - DELETE /api/alerts/:id (dismiss)
- ☐ Write integration tests for processing pipeline
- ☐ Performance testing (target: 100+ events/second)

Deliverables:

- ✅ Agent A (The Sieve) fully functional
- ✅ 90%+ spam filtering accuracy
- ✅ Alert generation system
- ✅ Alert APIs
- ✅ <5 second processing time per event

🔶 **Frontend Developer Tasks:**

Priority: P0 (Critical)

Week 5:

- ☐ Implement API service layer
    - Create axios instance with interceptors
    - Add JWT token to requests
    - Handle token refresh
    - Error handling and retry logic
- ☐ Integrate alerts API
    - Fetch user alerts (GET /api/alerts)
    - Pagination/infinite scroll implementation
    - Filter application
    - Loading and error states
- ☐ Implement Alert Detail Modal
    - Full event content display
    - Metadata section (source, timestamp, score)
    - Related alerts section
    - Action buttons (share, mark as read, dismiss)
    - Close and navigation controls

Week 6:

- ☐ Implement Settings page
    - Account settings tab
        - Email display

        - Password change form
        - Account deletion (with confirmation)
    - Notification settings tab
        - Channel toggles (Telegram, Email, Dashboard)
        - Quiet hours configuration
        - Alert frequency preferences
    - Alert preferences tab
        - Watchlist management (add/remove tokens)
        - Priority filter (which priorities to receive)
        - Save preferences button
- ☐ Create user preferences API integration
    - Save preferences to backend
    - Load preferences on mount
    - Real-time validation
- ☐ Add success/error notifications (toast)
- ☐ Implement loading states across the app

Deliverables:

- ✅ Alert feed with real data
- ✅ Alert detail view
- ✅ Settings page (fully functional)
- ✅ User preferences management

🔵 **Integration & DevOps Tasks:**

Priority: P1 (High)

Week 5:

- ☐ Integrate Telegram Bot API
    - Create Telegram bot
    - Implement /start command (account linking)
    - Generate and validate linking codes
    - Store telegram_id in user record
    - Implement /status command (show user stats)
    - Implement /settings command (quick prefs)
    - Implement /help command (bot instructions)
    - Implement /stop command (unsubscribe)
- ☐ Test Telegram bot functionality
    - End-to-end linking flow
    - Command responses
    - Error handling

Week 6:

- ☐ Integrate SendGrid Email API
  - Set up email templates
    - Welcome email
    - Email verification
    - Password reset
    - Alert email (HTML format)
  - Implement email sending service
  - Configure sender authentication (SPF, DKIM)
  - Test email delivery
- ☐ Implement alert delivery service
  - Create delivery worker (Celery)
  - Multi-channel delivery logic
  - Telegram message sending
  - Email sending
  - Delivery status tracking
  - Retry mechanism (3 attempts)
  - Fallback (Telegram fails → Email)
- ☐ Test alert delivery
  - Latency testing (Telegram <2s, Email <3s)
  - Success rate testing (target >99%)

Deliverables:

- ✅ Telegram bot operational
- ✅ Account linking functional
- ✅ Email service integrated
- ✅ Multi-channel alert delivery
- ✅ <2s Telegram delivery, <3s email delivery

Sprint 4: Week 7-8 - Real-time Features & WebSocket

🔷 **Backend Developer Tasks:**

Priority: P0 (Critical)

Week 7:

- ☐ Implement WebSocket server with Socket.IO
  - Set up Socket.IO with FastAPI
  - Connection authentication via JWT
  - Room-based communication (one room per user)

    - Handle connection/disconnection events
    - Heartbeat/ping-pong for connection health
- [ ] Create real-time alert broadcast system
    - Listen to alert generation events
    - Broadcast to connected users via WebSocket
    - Per-user alert filtering
    - Message queue integration
- [ ] Implement alert history endpoint
    - GET /api/alerts/history (last 30 days)
    - Pagination support
    - Filter by date, priority, entity
    - Search functionality (full-text search)
    - Export to CSV endpoint

Week 8:

- [ ] Implement admin dashboard API
    - System metrics endpoint (GET /api/admin/metrics)
        - Events ingested (hourly, daily)
        - Events processed
        - Alerts generated
        - Active users
        - System uptime
        - Error rate
    - User management endpoints
        - List users
        - View user details
        - Toggle user status (active/inactive)
- [ ] Add admin authentication middleware
    - Role-based access control (RBAC)
    - Admin-only routes
- [ ] Performance optimization
    - Database query optimization
    - Add indexes where needed
    - Connection pooling tuning
- [ ] Security hardening
    - Rate limiting implementation
    - Input sanitization review
    - SQL injection prevention (parameterized queries)
    - XSS prevention

Deliverables:

- ✅ WebSocket server operational
- ✅ Real-time alert delivery to dashboard (<1 second latency)
- ✅ Alert history with search and export
- ✅ Admin dashboard API
- ✅ Security hardening complete

🔶 **Frontend Developer Tasks:**

Priority: P0 (Critical)

Week 7:

- [ ] Integrate WebSocket with Socket.IO client
  - Connect to WebSocket server
  - Authenticate connection with JWT
  - Handle connection/disconnection
  - Automatic reconnection logic
  - Show connection status indicator
- [ ] Implement real-time alert updates
  - Listen for 'new_alert' events
  - Update alert feed in real-time
  - Smooth UI updates (no jarring insertions)
  - Show notification badge
  - Browser notification support (optional)
- [ ] Create Alert History page
  - List view of past alerts (last 30 days)
  - Date range picker
  - Priority and entity filters
  - Search functionality
  - Export to CSV button
  - Pagination controls

Week 8:

- [ ] Create Admin Dashboard (if admin user)
  - System metrics cards
    - Total events ingested
    - Total alerts generated
    - Active users
    - System uptime percentage
    - Error rate
  - Real-time metrics charts (using Chart.js or Recharts)
    - Events over time (line chart)

        - Alerts by priority (pie chart)
        - User growth (line chart)
    - User management table
        - List of users with status
        - Search and filter
        - Action buttons (view, disable)
- ☐ Implement user feedback mechanism
    - Feedback button on alerts
    - Simple thumbs up/down rating
    - Submit feedback form
- ☐ UI polish and refinements
    - Smooth animations and transitions
    - Loading skeletons
    - Error boundary implementation
    - 404 page
- ☐ Mobile responsiveness testing and fixes

Deliverables:

- ✅ Real-time dashboard updates
- ✅ Alert history page
- ✅ Admin dashboard (if admin)
- ✅ Fully responsive UI
- ✅ User feedback system

🔵 **Integration & DevOps Tasks:**

Priority: P0 (Critical)

Week 7:

- ☐ Set up production deployment environment
    - Choose hosting platform (Railway or Render)
    - Configure production database (PostgreSQL)
    - Set up managed Redis instance
    - Set up managed RabbitMQ instance (CloudAMQP)
- ☐ Create production Docker configuration
    - Multi-stage Docker builds
    - Optimize image sizes
    - Configure environment variables
    - Set up secrets management
- ☐ Set up CI/CD pipeline (GitHub Actions)
    - Automated testing on PR

    - Automated deployment on merge to main
    - Run linters and formatters
    - Build and push Docker images

Week 8:

- ☐ Deploy MVP to production
    - Deploy backend services
    - Deploy frontend
    - Configure domain and SSL/TLS
    - Set up HTTPS (Let's Encrypt)
- ☐ Set up monitoring and alerting
    - UptimeRobot for uptime monitoring
    - Sentry for error tracking and logging
    - Configure alert thresholds
    - Set up email alerts for critical errors
- ☐ Implement database backup strategy
    - Automated backups every 6 hours
    - 30-day retention policy
    - Test backup restoration
- ☐ Load testing and performance validation
    - Simulate 1,000+ concurrent users
    - Test 10,000+ events/hour ingestion
    - Validate <10 second end-to-end latency
- ☐ Security audit
    - SSL/TLS verification
    - OWASP Top 10 checklist
    - API security review
    - Rate limiting verification

Deliverables:

- ✅ Production environment operational
- ✅ CI/CD pipeline functional
- ✅ Monitoring and alerting set up
- ✅ Database backup strategy implemented
- ✅ MVP deployed to production
- ✅ Security audit completed

Phase 3: Testing, Polish & Launch (Weeks 9-12)

Sprint 5: Week 9-10 - Testing & Bug Fixes

🔷 **Backend Developer Tasks:**

Priority: P1 (High)

Week 9:

- ☐ Write comprehensive unit tests
  - Authentication flows
  - Event processing logic
  - Alert generation
  - API endpoints
  - Target: 80%+ code coverage
- ☐ Write integration tests
  - End-to-end user flows
  - Data ingestion → processing → alert delivery
  - Multi-user scenarios
  - Error scenarios
- ☐ Performance testing and optimization
  - Load testing with realistic scenarios
  - Database query optimization
  - Identify and fix bottlenecks
  - Memory leak detection

Week 10:

- ☐ Bug fixing based on testing results
  - Fix critical bugs (P0)
  - Fix high-priority bugs (P1)
  - Document known issues
- ☐ API documentation completion
  - Update OpenAPI/Swagger docs
  - Add request/response examples
  - Document error codes
  - Create API usage guide
- ☐ Code review and refactoring
  - Clean up technical debt
  - Improve code quality
  - Add missing docstrings
  - Optimize imports

Deliverables:

- ✅ 80%+ test coverage
- ✅ All P0 and P1 bugs fixed
- ✅ Complete API documentation
- ✅ Code quality improvements

🔶 **Frontend Developer Tasks:**

Priority: P1 (High)

Week 9:

- [ ] Write frontend tests
  - Component unit tests (Jest + React Testing Library)
  - Integration tests for key user flows
  - Test authentication flows
  - Test alert feed functionality
  - Test settings page
- [ ] Cross-browser testing
  - Chrome, Firefox, Safari, Edge (last 2 versions)
  - Fix browser-specific issues
- [ ] Accessibility audit
  - Run axe-core audit
  - Fix WCAG 2.1 Level A violations
  - Add ARIA labels where needed
  - Keyboard navigation testing
  - Screen reader compatibility testing

Week 10:

- [ ] UI/UX refinements based on testing
  - Improve error messages
  - Add contextual help/tooltips
  - Loading state improvements
  - Empty state designs
  - Success feedback improvements
- [ ] Performance optimization
  - Code splitting
  - Lazy loading components
  - Image optimization
  - Bundle size reduction
  - Lighthouse score optimization (target >80)
- [ ] Bug fixing
  - Fix UI bugs

    - Fix responsive design issues
    - Fix interaction bugs

Deliverables:

- ✅ Frontend tests (70%+ coverage)
- ✅ Cross-browser compatibility
- ✅ WCAG 2.1 Level A compliance
- ✅ Lighthouse score >80
- ✅ All UI bugs fixed

🔵 **Integration & DevOps Tasks:**

Priority: P1 (High)

Week 9:

- [ ] End-to-end integration testing
    - Test all data collectors with live APIs
    - Test alert delivery via Telegram
    - Test email delivery via SendGrid
    - Test WebSocket real-time updates
    - Test user onboarding flow
    - Test account linking (Telegram)
- [ ] Monitoring and alerting refinement
    - Fine-tune alert thresholds
    - Set up dashboards in monitoring tools
    - Create runbooks for common issues
    - Document incident response procedures
- [ ] Performance and scalability testing
    - Stress testing (>10,000 events/hour)
    - Concurrent user testing (1,000+ users)
    - Database performance under load
    - Identify scaling bottlenecks

Week 10:

- [ ] Disaster recovery testing
    - Test database backup restoration
    - Test service recovery procedures
    - Verify RTO and RPO targets
    - Document recovery procedures
- [ ] Security testing

    - Penetration testing (basic)
    - Test authentication bypass scenarios
    - Test SQL injection prevention
    - Test XSS prevention
    - Test CSRF protection
    - Rate limiting validation
- ☐ Infrastructure optimization
    - Cost optimization review
    - Resource utilization analysis
    - Scaling strategy documentation

Deliverables:

- ✅ All integrations tested and validated
- ✅ Monitoring dashboards operational
- ✅ Performance targets met (10k+ events/hr, 1k+ users)
- ✅ Disaster recovery plan tested
- ✅ Basic security testing complete

Sprint 6: Week 11-12 - Documentation, Launch Prep & Launch

🔷 **Backend Developer Tasks:**

Priority: P2 (Medium)

Week 11:

- ☐ Create comprehensive backend documentation
    - Architecture overview
    - Database schema documentation
    - API endpoint documentation
    - Event processing flow diagrams
    - Deployment guide
    - Troubleshooting guide
- ☐ Create developer onboarding guide
    - Local setup instructions
    - Contribution guidelines
    - Code style guide
    - Testing guidelines
- ☐ Final performance optimization pass
    - Profile slow endpoints
    - Optimize database queries

    - Review and optimize Celery tasks

Week 12:

- ☐ Pre-launch checklist
    - Verify all critical features working
    - Check all API rate limits
    - Verify data retention policies
    - Check backup procedures
- ☐ Launch day support preparation
    - Monitor system metrics
    - Be available for incident response
    - Prepare rollback procedures
- ☐ Post-launch monitoring
    - Watch error rates
    - Monitor performance metrics
    - Check alert delivery rates
    - Gather initial user feedback

Deliverables:

- ✅ Complete backend documentation
- ✅ Developer onboarding guide
- ✅ Pre-launch checklist completed
- ✅ Launch day support

🔶 **Frontend Developer Tasks:**

Priority: P2 (Medium)

Week 11:

- ☐ Create user-facing documentation
    - User guide / Help center
    - FAQ page
    - Getting started tutorial
    - Feature documentation
    - Video tutorial (optional)
- ☐ Final UI polish
    - Review all user flows
    - Ensure consistent design language
    - Fix any remaining visual issues
    - Add final animations/micro-interactions

- ☐ Create marketing assets
  - Screenshot high-quality dashboard views
  - Create feature demo GIFs
  - Update landing page with real screenshots
  - Social media preview images

Week 12:

- ☐ Pre-launch checklist
  - Test all user flows one final time
  - Verify mobile responsiveness
  - Check all links and navigation
  - Verify forms and validation
  - Test on multiple devices
- ☐ Launch preparation
  - Update landing page for launch
  - Add "Sign Up" CTA prominently
  - Ensure onboarding flow is smooth
- ☐ Post-launch monitoring
  - Monitor user feedback
  - Track user behavior (analytics)
  - Identify UX friction points
  - Quick bug fixes

Deliverables:

- ✅ User documentation and help center
- ✅ Marketing assets
- ✅ Final UI polish complete
- ✅ Pre-launch checklist completed
- ✅ Launch day support

🔵 Integration & DevOps Tasks:

Priority: P0 (Critical)

Week 11:

- ☐ Create operational runbooks
  - Common issues and solutions
  - Scaling procedures
  - Backup and recovery procedures
  - API key rotation procedures

    - Monitoring dashboard guide
- ☐ Set up analytics
    - Google Analytics or Plausible
    - User behavior tracking
    - Conversion funnel tracking
    - Custom events for key actions
- ☐ Final security hardening
    - Review all security headers
    - Verify HTTPS configuration
    - Check API rate limits
    - Review CORS settings
    - Verify input sanitization

Week 12:

- ☐ Launch day preparation
    - Final deployment to production
    - Verify all services running
    - Check all external integrations
    - Verify monitoring and alerting
    - Test emergency procedures
- ☐ Launch day execution
    - Monitor system health
    - Watch for errors and anomalies
    - Be ready for rapid response
    - Monitor API rate limits
    - Check alert delivery success rates
- ☐ Post-launch support (Week 12+)
    - Daily monitoring of key metrics
    - Respond to user issues
    - Scale resources as needed
    - Gather performance data
    - Plan for Phase II features

Deliverables:

- ✅ Operational runbooks
- ✅ Analytics tracking
- ✅ Final security review
- ✅ Successful MVP launch
- ✅ Post-launch monitoring and support

📊 Success Metrics & KPIs

Technical Metrics

Metric	Target	Critical Threshold
System Uptime	99.5%+	99.0%
End-to-End Latency	<10s (p95)	<15s
API Response Time	<500ms (p95)	<1s
Event Processing Throughput	10,000+/hr	5,000/hr
Alert Delivery Success Rate	>99%	>95%
Telegram Delivery Time	<2s	<5s
Dashboard Load Time	<2s	<3s
Database Query Time	<200ms (avg)	<500ms

User Metrics (Post-Launch)

Metric	Target	Measurement Period
User Registration	100+ users	First month
Onboarding Completion Rate	>70%	Ongoing
Daily Active Users (DAU)	40%+ of registered	After 2 weeks
Alert Engagement Rate	>60%	Ongoing
Telegram Linking Rate	>80%	Ongoing
User Retention (Week 1)	>50%	After launch

Quality Metrics

Metric	Target
Backend Test Coverage	80%+
Frontend Test Coverage	70%+
Critical Bugs (P0)	0 at launch

Metric	Target
High Priority Bugs (P1)	<3 at launch
Lighthouse Performance Score	>80
WCAG 2.1 Compliance	Level A

🚨 Risk Management & Mitigation

High-Risk Items

1. Twitter API Rate Limits

Risk: Twitter's basic tier may have strict rate limits
Mitigation:

- Implement robust rate limit handling
- Upgrade to higher tier if needed
- Have backup plan (reduce polling frequency)
- Monitor usage daily

2. Real-Time Processing at Scale

Risk: Processing 10,000+ events/hour may strain resources
Mitigation:

- Implement horizontal scaling early
- Use message queues effectively
- Monitor performance continuously
- Have auto-scaling triggers ready

3. Alert Delivery Failures

Risk: Telegram or SendGrid API failures could block alerts
Mitigation:

- Implement multi-channel fallback
- Retry logic with exponential backoff
- Queue system for failed deliveries

- Monitor delivery success rates

4. External API Dependencies

Risk: External APIs (Twitter, Ethereum node, etc.) may go down
Mitigation:

- Implement graceful degradation
- Use multiple providers where possible
- Cache data when appropriate
- Clear error messaging to users

5. Database Performance

Risk: Large event volume may slow down database
Mitigation:

- Proper indexing from start
- Implement data retention policies
- Regular performance monitoring
- Archival strategy for old data

🔄 Communication & Collaboration

Daily Standups (15 minutes)

Time: 10:00 AM daily
Format: Each person answers:

1. What did I accomplish yesterday?
2. What will I work on today?
3. Any blockers or dependencies?

Weekly Sprint Planning (1 hour)

Time: Monday, 2:00 PM
Agenda:

1. Review previous sprint

2. Plan current sprint tasks
3. Assign tasks and set priorities
4. Discuss dependencies and risks

Weekly Demo (30 minutes)

Time: Friday, 4:00 PM
Agenda:

1. Each person demos completed work
2. Gather feedback
3. Celebrate wins

Async Communication

- **Slack/Discord:** For quick questions and updates
- **GitHub Issues:** For bug tracking and feature requests
- **GitHub Projects:** For task management and sprint boards
- **Documentation:** For decisions and architectural choices

📝 Definition of Done

A task is considered "Done" when:

✅ Code is written and reviewed
✅ Unit tests written and passing (where applicable)
✅ Integration tests passing (where applicable)
✅ Code merged to main branch
✅ Documentation updated (if needed)
✅ Deployed to staging/production
✅ Smoke tested in deployed environment
✅ No critical bugs introduced

🎯 Phase II Preview (Post-MVP)

After successful MVP launch, the next phase (Months 4-6) will include:

1. **Agent B (The Profiler):** Wallet attribution and tracking
2. **Agent C (The Quant):** Advanced sentiment analysis
3. **Agent D (The Strategist):** Portfolio personalization
4. **Multi-chain support:** Solana, BSC, Layer 2s
5. **Mobile apps:** iOS and Android native applications
6. **B2B API:** Enterprise API access
7. **Advanced analytics:** Portfolio tracking and insights
8. **Premium features:** Automated trading signals (no execution)
9. **Social features:** Community sharing and discussions
10. **Subscription tiers:** Free tier limitations + Premium tier

---

🏁 Final Notes

Key Success Factors:

1. **Regular communication:** Daily standups and weekly demos
2. **Clear ownership:** Each person owns their domain
3. **Testing mindset:** Test early and often
4. **User focus:** Build for the end user, not just features
5. **Iterative approach:** Ship small, iterate fast
6. **Documentation:** Document as you go, not at the end

Critical Path Items:

The following items are on the critical path and must stay on schedule:

- Sprint 1: Authentication system (blocker for all user features)
- Sprint 2: Data ingestion (blocker for processing and alerts)
- Sprint 3: Event processing (blocker for alerts)
- Sprint 4: Alert delivery (core value proposition)
- Sprint 5: Testing (blocker for launch)
- Sprint 6: Launch preparation (final gate)

When Things Go Wrong:

1. **Identify blockers early:** Raise issues in daily standups
2. **Escalate quickly:** Don't wait days to ask for help
3. **Reprioritize if needed:** MVP scope can be adjusted

4. **Focus on core value:** Real-time crypto alerts are the priority
5. **Cut scope before quality:** Better to launch with fewer features than buggy ones

---

📞 External Resources & Support

API Providers Support

- **Twitter Developer Portal:** https://developer.twitter.com/en/support
- **QuickNode Support:** support@quicknode.com
- **Alchemy Support:** https://www.alchemy.com/support
- **Telegram Bot Support:** @BotSupport (Telegram)
- **SendGrid Support:** https://support.sendgrid.com/

Infrastructure Support

- **Railway:** https://railway.app/help
- **Render:** https://render.com/support
- **Sentry:** https://sentry.io/support/

Development Tools

- **FastAPI Docs:** https://fastapi.tiangolo.com/
- **React Docs:** https://react.dev/
- **PostgreSQL Docs:** https://www.postgresql.org/docs/
- **RabbitMQ Docs:** https://www.rabbitmq.com/documentation.html

---

END OF ROADMAP

Version: 1.0
Last Updated: January 2025
Next Review: End of Sprint 2 (Week 4)

Good luck with the build! Remember: Ship fast, iterate faster, and keep the user at the center of every decision. 🚀
"